Session 2: Staying in Control

Context, Parallel Sessions, Plan Mode, and Verification

Brady Allardice

UPF & IBEI

Where We Are

Session 1: What Claude Code is and how it works: permissions, models, context, and CLAUDE.md.

Session 2: Staying in control on a real research task.

Today you will:

1. Work within the context window, and run conversations in parallel
2. Build the county panel in plan mode, reviewing the plan before it runs
3. Learn the verification workflow: expectations first, tests that can fail
4. Learn the Swiss franc data, write your expectations, then merge and verify

The methods are straightforward. The skill is the workflow.

Start-of-Session Ritual

Work on a copy, not inside the shared repo. Do this now:

Mac / Linux

```
# Already cloned? Update:
cd ~/claude-code-workshop && git pull

# First time:
git clone https://github.com/
  bradyallardice/claude-code-workshop.git

# Copy and open
cp -r session_2/ ~/my_workspace/session_2/
cd ~/my_workspace/session_2
code .
```

Windows (PowerShell)

```
# Already cloned? Update:
cd ~\claude-code-workshop; git pull

# First time:
git clone https://github.com/
  bradyallardice/claude-code-workshop.git

# Copy and open
Copy-Item -Recurse session_2 `
  ~\my_workspace\session_2
cd ~\my_workspace\session_2
code .
```

Keep your `session_1` copy too: today's first exercise runs in it.

▸ git pull fails?

What Happens When the Context Window Fills Up

As a conversation grows, Claude Code eventually runs out of room.
When this happens, it **auto-compacts**: it summarizes older parts of the conversation to free up space.

What auto-compacting does:

- Replaces earlier messages with a compressed summary
- Keeps recent messages and tool results intact
- Happens automatically. You will see a notice in the chat

The risk: Details from early in the conversation (variable definitions, decisions you discussed, constraints you mentioned) may be lost or simplified in the summary.

You can also compact manually:
Type `/compact` to force a summary and free up context space.

This is why CLAUDE.md matters: it survives compacting because it is reloaded from disk, not from conversation history.

Strategies for Working Within the Context Window

Recall from Session 1: Claude Code can only "see" a limited amount of text at once. Errors compound when it loses track of earlier decisions.

Five strategies to work within this constraint:

1. Keep tasks focused
2. Maintain project documentation
3. Summarize work between conversations
4. Validate iteratively
5. Write token-aware prompts

Strategy 1: Keep Tasks Focused

One topic, one conversation.

Avoid:

- "Clean the data, build the panel, run the regressions, and make tables"
- Long conversations that drift across topics
- Asking Claude Code to hold your entire project in its head

Instead:

- "Clean and reshape the raw census file"
- Start a new conversation for each distinct task
- Let each conversation do one thing well

Why

A focused task fits comfortably in the context window. A sprawling task forces Claude Code to forget earlier steps, and that is where silent errors enter.

What Counts as One Task?

A task is a unit of work with **one deliverable and one check**.

One task:

- "Reshape the census file to county-year"
- "Merge the survey onto the panel"
- "Run the baseline regression"

Several tasks in disguise:

- "Clean the data and run the regressions"
- "Get the analysis working"
- "Fix the merge and rename variables"

Three tests for a single task:

- You can state it in one sentence
- It produces one output you can inspect: a cleaned file, a table, a figure
- You know in advance how you will check that it is right

The check is the boundary

If you would want to inspect something halfway through (does the merge look right?), that point separates two tasks.

One Conversation per Task

Start a new conversation when

- The task is finished and you have checked its output
- You switch to a different dataset, file, or question
- Claude contradicts an earlier decision or asks again for something you already said
- You see the auto-compact notice

What carries over

- **Does not:** anything said only in the chat, such as variable definitions, decisions, constraints
- **Does:** files on disk: scripts, `CLAUDE.md`, your notes

A new conversation is not starting over

The project is still on disk. Before you close a conversation, write down anything the next one needs.

Multiple Claude Code Tabs

You can run **several conversations at once**. Each one has **its own history and context**.

How to open another one

- **Open in New Tab**: `Cmd+Shift+Esc` (Mac), `Ctrl+Shift+Esc` (Windows)
- Also in the Command Palette: **Open in New Window**, **Open in Side Bar**
- Drag the panel to either sidebar or the editor area

A useful pattern

- Main task in the sidebar
- Side tasks in separate tabs, e.g. "explain this script" while the main one is busy

Give each tab a different job

Two tabs editing the same file can step on each other. Split the work by file, and keep the same conversation open in only one place.

Keeping Track of Your Conversations

Status dot on the Spark icon

- **Blue:** a permission request is waiting for you
- **Orange:** Claude finished while that tab was hidden

So you can leave a long task running in one tab and still notice when it needs you.

Finding old conversations

- **Session history** button at the top of the Claude Code panel
- Search by keyword or browse by time
- Clicking one resumes it, or jumps to its tab if it is already open

A permission request that sits unseen in a hidden tab stalls that task. Check the dot.

Demo: Two Tabs at Once

Live demo — follow along on your own machine

Open your `session_2` copy in VS Code. Open two Claude Code tabs, paste one prompt into each, and send both.

Tab A: one narrow task

```
Read docs/codebook.md and list the variables in
data/swiss_franc_survey.csv with a one-line description
of each. Do not modify any files.
```

Tab B: a different narrow task, at the same time

```
Read data/swiss_franc_survey.csv and report how many
respondents are in each treatment group. Do not modify
any files.
```

Two independent, read-only tasks: each tab has its own clean context, and neither can disturb the other.

Demo: The Collision Round

Watch this one — don't run it yourself

Same two tabs. This time both write to **the same file**.

Tab A

```
Write the variable list you just made to output/data_notes.md.
```

Tab B

```
Write the treatment-group counts you just made to
output/data_notes.md.
```

Watch for: whose content is in the file at the end? Did either tab notice that the other had changed it?

Files are the only memory the tabs share. When two sessions write the same file without checking, the last writer wins.

Parallel Sessions: Benefits and Risks

Benefits

- **Throughput** on independent tasks
- **A clean context** for each task
- **A fresh session is an independent checker:** it has not seen the other's reasoning

Risks

- **No shared memory except files.** A tab already open loaded CLAUDE.md when it started, so it won't see later edits unless it re-reads the file
- **Collisions:** two sessions write the same output, and the last writer wins
- **Split attention:** you review several streams of output at once
- **Stalls:** a hidden tab can sit waiting on a permission prompt
- **Cost:** token use scales with the number of sessions

The same holds for several ChatGPT or claude.ai chats: each one knows only what you gave it.

Strategy 2: Maintain Project Documentation

Claude Code reads certain files **automatically** every time it starts.

Keep these up to date:

- `CLAUDE.md`: conventions, constraints, variable definitions
- `README.md`: project structure, what each directory contains
- Directory-level `CLAUDE.md` files for subfolders with specific rules

The Payoff

Good documentation means Claude Code starts every conversation already knowing your conventions, without you spending tokens re-explaining them.

Live Demo: What Claude Code Looks Like in Practice

I'll switch to VS Code and show you Claude Code working on a real project of mine: AI Adoption Prediction, a project on how well sociodemographics predict individual AI adoption across countries.

The project:

- Research folder: `src/` (scripts), `data/`, `docs/`, `outputs/`, a README

What to watch for:

- Where Claude Code sits in the VS Code window
- How it **reads** files in the project folder
- How it **proposes edits** as a diff
- How you **approve, reject, or redirect** each change

Nothing to follow along with here. Just watch; you'll do this yourself in the exercise.

Starting a New Project in VS Code

VS Code has no "New Project" button. **A project is just a folder** you open.

1. **Create an empty folder** in Finder / File Explorer (or use **New Folder** inside the Open Folder dialog)
2. **Open it:** File → Open Folder
3. **Trust the folder** when VS Code asks. The Claude Code extension does not work in Restricted Mode
4. **Build the structure** in the Explorer panel: right-click → New Folder / New File
5. **Open Claude Code** in that window

A typical research layout:

```
my_project/
  README.md
  src/  scripts
  data/
  docs/
  outputs/
```

From a terminal, `code .` opens the current folder in VS Code.

One project = one folder = one VS Code window. Claude Code works on the folder you have open.

Strategy 3: Summarize Work Between Conversations

When you finish a conversation, the context is gone. The next conversation starts fresh.

Before ending a conversation:

1. Ask Claude Code to summarize what it did and what remains
2. Save that summary (in a file, a commit message, or your notes)
3. Paste the summary into the *next* conversation as context

Example prompt:

```
Summarize what we did in this session: what files
were changed, what decisions were made, and what
still needs to be done.
```

Think of it like a handoff memo between research assistants.
The next "assistant" knows nothing unless you brief it.

Demo: Summarizing a Session

Live demo — just watch

I ask Claude Code for a handoff summary at the end of a real working conversation.

What I will do:

1. Ask for a summary of what was done and what remains
2. Save it to a file
3. Use it to start the next conversation

Strategy 4: Validate Iteratively

The Compounding Error Problem

If step 1 is wrong and you do not catch it, steps 2–5 build on that mistake. By the time you check the final output, the error is buried.

Check each step before moving to the next:

1. Clean the data → *verify row counts and distributions*
2. Construct variables → *spot-check values against known cases*
3. Run the merge → *confirm pre- and post-merge row counts*

Slower per step, but **much faster overall** because you never have to unwind a chain of errors you didn't catch.

We practice this in the validation section later today.

Strategy 5: Token-Aware Prompting

Every word in your prompt costs tokens. Make them count.

A good prompt tells Claude Code three things:

1. **What files matter:** "Read `data/raw/census.csv` and `scripts/clean.py`"
2. **What the goal is:** "Create a county-year panel with one row per county per decade"
3. **What constraints apply:** "Keep all 3,143 counties; do not drop rows with missing population"

Vague:

"Clean up the data and make it ready for analysis"

Specific:

"Read `census.csv`, drop rows where `year` < 1950, and save as `panel_clean.csv`"

The Rule

Be specific about *inputs*, *outputs*, and *constraints*. Be brief about everything else. Context is finite; spend it on what matters.

Demo: A Good Prompt

Live demo — just watch

I give Claude Code the same task twice: a vague prompt, then one that names the files, the goal, and the constraints.

What to look for:

- Which files does the prompt point Claude Code to?
- What is the goal, stated in one sentence?
- What constraints are spelled out?

The Core Workflow

	Step	Who	If something is wrong
1.	Describe	You	Rethink before sending
2.	Review plan	You *	Reject, re-instruct
3.	Generate & validate	Claude Code + You	Read the code, run the tests, re-instruct
4.	Save or re-instruct	You	Keep the result (git in Session 3)

* = checkpoint where you can catch a problem before anything gets saved.

Validation is not a separate step — it happens **continuously** throughout steps 1–3.

The pattern

When something goes wrong, go back to **whichever step actually fixes the problem**: a bad plan means rewriting the instruction (step 1), not just re-running the same one. Diagnose first, then go back to the right step.

Step 1: Describe — Write It Down First

Before you type anything into Claude Code, write down your instruction.
If you cannot state the task precisely on paper, Claude Code cannot execute it precisely in code.

A good instruction specifies three things:

1. **Inputs:** "Read `data/survey.csv` and `data/demographics.csv`"
2. **Goal:** "Left-join demographics onto the survey by respondent ID"
3. **Constraints:** "Output must have exactly 2,044 rows. Do not drop respondents."

Vague:
"Merge the datasets and run a regression"

Specific:
"Left-join demographics onto the survey by respondent_id. Use control as the reference. Run a weighted logistic regression."

Be specific about inputs, outputs, and constraints. Be brief about everything else.

The Three Execution Modes

The most important configuration decision you will make.

Plan Mode: Reviews approach with you *before* acting

Default Mode: Acts, but asks permission for "significant" steps

Auto-Accept: Executes everything without asking

↓ More autonomy, less oversight ↓

How it works:

- Claude Code analyzes your request
- Proposes a detailed plan (files to read, changes to make, approach)
- Waits for your explicit approval before acting
- You review, approve, modify, or reject

When to use:
Any task where the *approach* matters, not just the output.
Data transformation, variable construction, merges, anything analytical.

Why It Matters

The plan is your checkpoint. This is where you catch "I am going to do an inner join" *before* it happens, not after 300 observations are gone.

How it works:

- Claude Code executes steps autonomously
- Pauses for actions *it* considers significant
- Uses its own judgment about when to ask

When to use:
Routine tasks in established projects where you trust the general approach.

The Risk for Academics

Claude Code's judgment about what is "significant" may not match yours.
It might pause before writing a file but **not** before silently recoding a variable.

How it works:

- Executes everything without asking
- Reads, writes, runs, and modifies freely
- Stops only on errors

When to use:

- File reformatting
- Boilerplate generation
- Documentation

When not to use:
Anything involving your data or analysis.
The speed gain is not worth the loss of checkpoints.

The Key Insight on Modes

Different Relationships with the Agent

Plan mode = you **direct** the work
Auto-accept = you **review** the work after the fact

For published research, directing is almost always safer than reviewing after the fact.

Why: Errors you *prevent* at the planning stage are cheaper than errors you *catch* in the output, or worse, errors you don't catch at all.

Live Demo: Plan Mode vs. Auto-Accept

Same task, two modes.

Watch for:

- What does a plan look like?
- How do you evaluate it?
- What does the experience feel like in each mode?

Step 2: Review the Plan

In plan mode, Claude Code proposes an approach before acting.

What to look for:

- Does it read the right files?
- Does the merge/analysis strategy match what you specified?
- Does it handle edge cases (missing data, extra rows, reference categories)?
- Does it make any *analytical decisions* you did not ask for?

The key question

Is there anything in this plan that I did not explicitly ask for?
If yes: reject or modify before it runs.

You do not always need plan mode. For simple tasks — summary statistics, reformatting, boilerplate — steps 1–3 collapse into one. But when uncertain, *use it*.

Step 3: Generate and Validate

Claude Code writes and runs the code. Your job is not done.

Read what Claude Code produced:

- Does the script do what the plan said it would?
- Are variable names and file paths correct?
- Did it make any choices the plan did not mention?

Common surprises at this stage:

- Silently dropping rows with missing values
- Choosing a default reference category you did not specify
- Using unweighted estimation when you asked for weighted
- Adding "helpful" data cleaning steps you did not request

The habit

Read the code, not just the output. Output can look correct even when the code is wrong.

Step 4: Save or Re-instruct

If the output passes verification: keep it. Claude Code saves its changes to your files after each step, so the results are already on disk.

If something is wrong: identify the specific error and re-instruct. Do not ask Claude Code to "fix it" — tell it *what* to fix and *how*.

In Session 3 we will learn **git** — a tool that lets you snapshot your project before each change and revert if anything goes wrong. For now, know that the save step exists and that there is a better way to manage it coming soon.

The full cycle

Describe → Review plan → Generate & validate → Save or re-instruct

Repeat for each task in your analysis. One cycle per task, not one cycle for the whole project.

Exercise: Build the County Panel From Scratch

In Session 1 you explained and edited a pre-built script. Now **you** build it.

Same data, fresh start, in your `session_1` copy. Work in pairs using plan mode.

1. Write a short `CLAUDE.md` for the folder (template: `references/CLAUDE_template.md`)
2. Switch Claude Code to **plan mode**
3. Ask Claude Code to build a county-year panel that merges election returns with IPUMS census demographics
4. **Before approving the plan**, you will need to:
   - Read the codebooks to understand variable codings
     (What does `EDUC` $\geq$ 10 mean? What does `RACE == 1` code?)
   - Decide how to define key variables
     (Two-party vote share? College = bachelor's or higher?)
   - Specify the merge: which keys, which join type, expected row counts
5. Review the plan, reject or revise as needed, then approve
6. Run the script and verify the output

Evaluating the Plan

When Claude Code proposes its plan, ask yourself:

- Is it reading the **codebooks** or just guessing at variable meanings?
- How is it coding `EDUC`, `RACE`, `HISPAN`?
  Do those match *your* definitions?
- What join type is it using? Inner, left, or outer?
  What happens to counties that appear in one dataset but not the other?
- Is it filtering `mode == "TOTAL"` for election data?
- Does it handle `COUNTYFIP == 0` (unidentified counties)?

This is where your domain knowledge matters.
Claude Code can write the code, but you decide what the variables mean.

Debrief

- What variable coding decisions did you make? Did Claude Code's default match yours?
- Did anyone reject or modify the plan? What did you change?
- How did your panel compare to the starter script from Session 1?

The Lesson

Writing a CLAUDE.md forces you to be explicit about conventions you have kept implicit.

Reviewing plans forces you to think about approach *before* you see output.

Domain knowledge is key for preventing errors.

Verification: Code Can Run and Still Be Wrong

Claude Code can write code that runs, prints a plausible-looking table, and is still **scientifically wrong**.

None of these raises an error:

- A merge that duplicates rows when keys repeat
- A regression that silently drops every row with a missing control
- Standard errors clustered at the wrong level
- A reference category chosen alphabetically instead of by design

Two different questions

Software tests ask: does the code run and behave as intended?
Research tests ask: are the numbers right?

Read the Code

Claude Code is not deterministic — the same prompt can produce different code each time. But **code itself is deterministic.** If you read the code and understand what it does, you can be confident in the results, the same way you would be with code you wrote yourself.

This is the most fundamental verification:

- Read the script Claude Code produced, line by line
- Confirm it does what you asked — not more, not less

The principle

You are not trusting Claude Code. You are trusting *the code*. Reading does not scale to every number, which is why we also write tests.

Layer 1: Software Tests Check That the Code Works

Test	It asks	Example from today's data
Smoke	Does the project run at all?	Packages import; both CSV files load
Unit	Does one function behave correctly?	The recode returns 1 for "some" or "big" and 0 for "none"
Integration	Do the pieces fit together?	The cleaned data has every column the model script uses
End-to-end	Does the whole pipeline run?	Raw data to final table from a single command
Regression (software sense)	Did a change alter verified results?	After refactoring, N and coefficients are unchanged
Edge-case and negative	Do unusual or bad inputs fail loudly?	A duplicate ID stops the merge instead of multiplying rows

Layer 2: Research Tests Check That the Results Are Right

Test	It asks	Example from today's data
Data integrity	Are keys, types, and ranges valid?	`respondent_id` is unique; age is between 18 and 85
Merge integrity	Did the join keep and match what it should?	2,044 rows in and out; unmatched respondents are counted
Sample accounting	Where did every row go?	N for each model, with a reason for every drop
Variable construction	Does each variable match its definition?	`supports_intervention` is 1 exactly when the answer is "some" or "big"
Estimation specification	Is the model the one you intended?	Control is the reference arm; survey weights are used
Independent numerical check	Does a separate calculation agree?	A weighted share recomputed straight from the raw CSV

Independent Tests

A test only helps if it can fail for a reason the code cannot also fail for.

Independent:
Expected values come from outside the code under test: computed by hand from the raw file, by a different method, or from a published result.

Not independent:
Running the pipeline and pasting its output in as the "expected" value. The test can only confirm that the code agrees with itself.

Example prompt:

```
Write and run tests for these values, which I computed
from the raw file:
- Respondent 9 has treatment = "cntrl", fx_status = "none"
- The control group has exactly 480 respondents
- The weighted share supporting intervention, among
  non-missing responses, is approximately 0.486
Do not take expected values from the analysis script.
```

Write Down Expectations First, Then Assert Them

The Pre-Registration Analogy

Just as you pre-register hypotheses before seeing results, write down what a correct transformation should produce *before* you run it.

Before a merge or transformation, state:

- Expected rows $\times$ columns
- Which columns, and their types
- How missing values are handled
- A row you can trace by hand

Then assert what must be true:

- A left join keeps every row of the left table
- A new control changes N only through missing values
- A share lies between 0 and 1
- Treatment arms sum to the total sample

A mismatch is either a bug or a misunderstanding. Both are worth catching early.

Trace Samples End to End

Aggregate checks can pass while individual records are wrong.
Follow specific records through every stage of the pipeline.

Raw file → Merged → Recoded → Estimation sample → Table

- Choose 3–5 respondents *on purpose*: typical, missing value, no match in file 2
- Print each one's values at every stage, side by side
- Tie every number in the final table (N, coefficients) to the rows behind it

Example prompt:

```
Pick one typical respondent, one with a missing outcome, and
one absent from respondent_demographics.csv. Show each one's
row in the raw, merged, and regression data.
```

The Test-First Cycle

Repeat for every component: merge, variables, model, table.

1. **You state the expectations:** files, keys, row counts, edge cases
2. **Claude Code writes the tests first.** You read them before any analysis code exists
3. **It implements the component** and runs the tests
4. **Failures are investigated, not patched:** fix the code, not the expected value
5. **Re-run the earlier tests** before moving to the next component

Why tests first

Tests written after the code tend to encode what the code does. Tests written first encode what *you* expect.

Test the Tests

A test that cannot fail is not a test. Inject a known error and confirm the check catches it.

Fault injection (always on a copy, never the raw data):

- Duplicate one survey respondent's row in a copy of the demographics file: the row-count and key tests should fail
- Change one treatment label in a copy: the spot check should fail

Planted answer: simulate outcomes with a known treatment effect. The estimator should recover it within a stated tolerance.

Example prompt:

```
Copy respondent_demographics.csv with one survey
respondent's row duplicated. Run the merge tests on the
copy and show me which tests fail.
```

Passing Is Not the Goal

Claude Code must never make a test pass by:

- Editing expected values to match the output
- Deleting or loosening assertions
- Suppressing warnings
- Changing the specification: join type, reference category, sample, weights

Not every check is pass/fail:

- **Hard failure** (duplicate keys): stop
- **Warning** (missingness jumps from 4% to 11%): flag it for you
- **Judgment** (is identification credible?): yours

A failing test is information

Claude Code should report it and ask. It should not make it disappear.

Rules in CLAUDE.md Come From Failures

`CLAUDE.md` holds standing instructions: Claude Code reads it at the start of every conversation. A rule is worth writing only when you know the failure it prevents.

A good rule has three parts:

1. The failure you saw
2. The rule, in your own words, specific enough to follow
3. How you would notice that it was broken

Failures to watch for in Part 1:

- Tests whose expected values were copied from the code's own output
- A design choice (the join type) made for you, without asking
- A failing test "fixed" by changing the test
- "All tests passed" with no word on what was not tested

Do not write rules yet. After Part 1 you write your own, from what actually happened to you.
`CLAUDE.md` is an instruction, not enforcement: read the tests too.

Today's Data: The Swiss Franc Shock

Background: In January 2015, the Swiss National Bank removed its currency peg. The Swiss franc surged against the Polish zloty. Polish households with CHF-denominated mortgages saw their payments spike overnight.

The survey: 2,044 Polish adults, interviewed October 2015. Respondents were randomly assigned to receive different information about the crisis before being asked whether the government should intervene to help affected borrowers.

Your files:

- `swiss_franc_survey.csv` — 2,044 respondents, 11 variables
- `respondent_demographics.csv` — income and ideology data (more rows than the survey)
- `codebook.md` — variable descriptions

Source: Ahlquist, Copelovitch, and Walter, "The Political Consequences of External Economic Shocks: Evidence from Poland," *American Journal of Political Science* (2020).

The Research Question

Does providing information about the Swiss franc crisis increase support for government intervention to help affected borrowers?

Treatment (randomized):

- Control (no info)
- Factual information
- Historical context
- Hungary's policy response

Outcome:

- Supports government intervention (binary)

Moderator:

- FX loan exposure status

The methods are straightforward. The skill is staying in control of the workflow.

Exercise Part 1: Learn the Data, Then Merge and Verify (35 min)

Learning the data is the part you never delegate. Write down what a correct merge must look like *before* Claude Code touches the data, then check its work against your list.

0–10 min	Step 1: learn the data and write your expectations, **without Claude Code**
10–22	Step 2: merge and explore in plan mode; Claude Code writes the tests from your expectations
22–27	Step 3: verify the output against your expectations
27–31	Debrief
31–35	Step 4: write your own verification rules in `CLAUDE.md`

In Session 4 you will see how to hand the *checking* to Claude. The learning stays with you.

Part 1, Step 1: Before Claude Code Touches the Data

1. **Learn the data and write your expectations, without Claude Code.** Read `docs/codebook.md` and open both CSV files. Then write `expectations.md` in your `session_2` folder:
   - How many rows should the merged file have, and which key links the two files?
   - Which join type, and why? What happens to survey respondents with no demographics row?
   - Which variables have missing values, and how many do you expect?
   - What does `income_quintile` $= 0$ mean, and how must it be treated?
   - For each answer: how would you check it?

These are your expectations. Claude Code writes its tests from this file.

Part 1, Steps 2–3: Merge, Then Check It Against Your List

2. **Merge** the demographics data into the survey, in plan mode. Save it as `output/merged_survey.csv`.
   - Tests first: *"Write the merge tests from expectations.md."* Read them before it writes the merge.
   - You decide the join type. Do not let Claude Code choose.
   - Then explore: summary statistics, treatment group sizes, outcome distribution.

3. **Verify it yourself.** Go down `expectations.md` and mark each item met or not met, with the evidence.
   - Run the tests. Trace three respondents: typical, missing outcome, absent from the demographics file
   - Test the tests: in a *copy* of the demographics file, duplicate the row of one **survey respondent**. The merge tests should fail.

You have no `CLAUDE.md` rules yet. Notice where Claude Code cuts a corner. You will write your rules from that.

Part 1 Debrief: What Did Your Expectations Get Wrong?

- Which of your expectations turned out to be wrong? Was the error yours, or Claude Code's?
- What did you learn about the data that the codebook did not say outright?
- Did Claude Code's tests check what you wrote, or something easier?
- Did the tests fail on the copy with a duplicated survey respondent?

The point

You can only verify work against expectations you hold. Writing them down is how you learn the data, and that part is not delegated.

Part 1 Debrief: What Did You Verify?

What did you check?

Part 1 Debrief: What Did You Verify?

What did you check?

You should have verified:

- Row count: exactly 2,044 rows (no rows gained or lost)
- Join type: only a left join gives the correct result
- No duplicate respondent IDs in the output
- Treatment groups: 480 / 549 / 520 / 495
- Proportion supporting intervention: $\sim$0.479 (unweighted)
- FX exposure: 1,918 / 69 / 55
- Test the tests: the duplicated row made the row-count and key tests fail

Did your numbers match? Did Claude Code choose the right join type?

Part 1, Step 4: Write Your Own Rules

Create `CLAUDE.md` in your `session_2` folder. Write **3–5 rules, in your own words**, each from something that happened in Part 1.

For each rule, write:

1. **Failure:** what happened, or nearly happened
2. **Rule:** one specific instruction to Claude Code
3. **Check:** how you would notice it was broken

The shape of one (not one to copy)

Failure: a test expected N to equal the length of the script's own output. *Rule:* expected values come from `expectations.md` or the raw files, never from the script. *Check:* a hard-coded number that matches the output to the last digit.

Rules take effect in a new conversation. Keep the file: Session 3 builds on it and compares it with a reference.

Next Session

Session 3: Verification Through to the Paper

- Estimate the models and account for every observation, then robustness checks and a LaTeX table
- Git as the undo button: commit → instruct → diff → revert
- LaTeX in VS Code: from results to a paper that Claude Code can edit and check

Before next session:

- Keep your `my_workspace/session_2` folder as it is. Session 3 continues the exercise in it.
- Make sure `setup_check.tex` from Session 1 compiles (setup guide, Step 7)
- Try the core workflow on something from your own research, and note where Claude Code made a decision you did not ask for

Back Matter

Block	Min
If `git pull` fights back	2

If `git pull` Fights Back

If you committed work inside the repo, `git pull` will complain. Run this recovery in the course repo, then continue with Steps 2–3:

```
git stash -u                        # save uncommitted edits
git branch backup-prework           # save any local commits
git fetch origin
git reset --hard origin/student     # match the server
git stash pop                       # skip if stash was empty
```

Your old commits live on `backup-prework`. If that name exists, add a suffix: `backup-prework-2`.

Back to the ritual

Back matter